

■ Guía Completa de Herramientas

SeenLabs · Claude Code · Workflow de Desarrollo IA

Versión 1.0 · Mayo 2026 · gabo.93.gl@gmail.com

1. ¿Qué es todo esto y por qué importa?

Esta guía documenta el stack completo de herramientas de desarrollo asistido por IA instaladas en tu entorno Claude Code para el proyecto SeenLabs Webpage. No es un simple listado de comandos: es una metodología de trabajo que transforma cómo construyes software.

La idea central: En lugar de pedirle a Claude que escriba código directamente ("vibe coding"), primero defines QUÉ quieres construir con especificaciones precisas, luego planificas CÓMO hacerlo, y solo entonces implementas. El resultado es código más predecible, seguro y mantenible.

2. Resumen del Stack Instalado

Herramienta	Tipo	Alcance	Qué hace en una línea
security-guidance	Plugin	Proyecto	Revisa vulnerabilidades en cada edición y commit
superpowers	Plugin	Proyecto	Fuerza diseño → plan → TDD antes de implementar
claude-mem	Plugin	Global	Construye memoria pasiva del proyecto entre sesiones
spec-kit (specify)	CLI + Skills	Proyecto	Workflow estructurado: spec → plan → tareas → impl
ui-ux-pro-max	Skill	Global	Diseño UI/UX con 67 estilos, 96 paletas, 13 stacks
Skills de Engineering	Skills	Global	Debug, arquitectura, deploy, testing, etc.
Skills de Marketing	Skills	Global	Contenido, SEO, campañas, brand review
Skills de Archivos	Skills	Global	PDF, Excel, Word, PowerPoint

3. Plugins del Proyecto

3.1 security-guidance

¿Qué es? Un plugin oficial de Anthropic que actúa como un revisor de seguridad automático que nunca duerme. Analiza el código que Claude escribe en tiempo real y lo corrige en la misma sesión, antes de que llegue a un pull request.

Las 3 capas de protección:

Capa	Cuándo actúa	Cómo funciona	Costo
Per-edit (por edición)	Cada vez que Claude escribe en un archivo	Pattern matching instantáneo. Busca strings peligrosos: eval(), os.system(), innerHTML=, pickle, dangerouslySetInnerHTML, .github/workflows/	Cero — sin llamada al modelo
End-of-turn (fin de turno)	Al terminar cada respuesta de Claude	Calcula un git diff de TODO lo que cambió en el turno (ediciones, bash, subagentes) y lo envía a un modelo de revisión separado con contexto de seguridad	Una llamada al modelo por turno
Commit review (en cada commit)	Cuando Claude hace git commit o git push	Revisión agéntica profunda: lee código circundante, callers, sanitizers y archivos relacionados para determinar si un hallazgo es real o un falso positivo	Varias llamadas; máx 20/hora

Vulnerabilidades que detecta (capa modelo):

- Authorization bypass — rutas que saltan autenticación
- Insecure Direct Object References (IDOR) — acceso a recursos sin verificar permisos
- SQL/Command/LDAP Injection — datos de usuario insertados sin sanitizar
- Server-Side Request Forgery (SSRF) — el servidor hace requests controlados por el atacante
- Weak cryptography — MD5, SHA1, claves hardcodedas, IV fijos
- DOM Injection / XSS — escritura directa al DOM sin escape
- Unsafe deserialization — pickle, eval, exec con datos externos

Nota importante: security-guidance NO bloquea escrituras ni commits. Reporta hallazgos como instrucciones a Claude, quien los resuelve en la conversación. Es una capa de defensa en profundidad, no un firewall.

Variables de control:

Variable	Efecto
ENABLE_PATTERN_RULES=0	Desactiva el pattern matching per-edit
ENABLE_STOP_REVIEW=0	Desactiva la revisión end-of-turn
ENABLE_COMMIT_REVIEW=0	Desactiva la revisión de commits
ENABLE_CODE_SECURITY_REVIEW=0	Desactiva todas las revisiones con modelo
SECURITY_GUIDANCE_DISABLE=1	Desactiva el plugin completo sin desinstalarlo
SECURITY_REVIEW_MODEL=c laude-opus-4-8	Cambia el modelo de revisión end-of-turn

3.2 superpowers

¿Qué es? Una metodología completa de desarrollo de software para agentes de IA, construida sobre skills composables. Evita que Claude empiece a escribir código sin entender bien el problema. Impone un flujo de trabajo disciplinado: primero entender, luego diseñar, luego implementar.

Lo que cambia con superpowers activo:

- **Diseño antes de código:** Claude hace preguntas, presenta un diseño para tu aprobación, y solo implementa cuando das el visto bueno.
- **Planeación estructurada:** Las tareas se dividen en chunks pequeños con especificaciones exactas antes de comenzar.
- **TDD forzado:** Ciclos RED-GREEN-REFACTOR — primero se escribe el test que falla, luego el código que lo pasa, luego se refactoriza.
- **Debugging sistemático:** Proceso estructurado de análisis de causa raíz en lugar de trial-and-error.
- **Code review automático:** Revisión de calidad y cumplimiento integrada al flujo.

3.3 claude-mem

¿Qué es? Un plugin de memoria persistente que observa cómo trabajas con Claude y construye un modelo mental del proyecto. En cada nueva sesión, inyecta automáticamente el contexto relevante para que Claude no empiece desde cero.

Cómo funciona:

- Observa pasivamente: qué archivos editas, qué comandos corres, qué patrones usas.
- Almacena todo en ~/.claude-mem en tu máquina (no sale a la nube).
- En tu segunda sesión en adelante, inyecta contexto relevante automáticamente al inicio.
- Puedes ver las observaciones en tiempo real en <http://localhost:37702>.
- El worker debe estar corriendo: `npx claude-mem start` (en background).

Comandos útiles:

```
npx claude-mem start # Inicia el worker (hacer cada sesión)

npx claude-mem stop # Detiene el worker

npx claude-mem status # Ver estado del worker

/learn-codebase # Ingesta todo el repo de golpe (~5 min, opcional)

/how-it-works # Explicación del sistema de memoria
```

Importante: claude-mem está instalado globalmente (en ~/.claude/settings.json), por lo que funciona en TODOS tus proyectos, no solo en SeenLabs Webpage.

4. Spec-Kit: Desarrollo Guiado por Especificaciones

¿Qué es? Un toolkit open-source de GitHub que implementa Spec-Driven Development (SDD). La idea es invertir el flujo tradicional: en lugar de codear y luego documentar, primero escribes especificaciones ejecutables y de ellas se genera la implementación. Instalado como CLI (specify) y como skills locales en `.claude/skills/`.

4.1 Flujo Principal (paso a paso)

Paso	Skill	Descripción detallada
1	<code>/speckit-constitution</code>	SOLO UNA VEZ POR PROYECTO. Establece los principios fundamentales: stack tecnológico, convenciones de código, restricciones de arquitectura, estándares de calidad. Es la "constitución" que guía todas las decisiones futuras.
2	<code>/speckit-specify</code>	El paso más importante. Describes en lenguaje natural qué quieres construir. Claude genera una especificación estructurada con: casos de uso, comportamiento esperado, criterios de aceptación, y lo que ESTÁ FUERA del scope.
3	<code>/speckit-clarify</code> (opcional)	Si la spec tiene áreas ambiguas, este skill genera preguntas estructuradas para de-riesgar el proyecto antes de planear. Úsalo cuando la funcionalidad sea compleja o el scope no esté del todo claro.
4	<code>/speckit-plan</code>	Genera el plan técnico de implementación: arquitectura de componentes, decisiones de diseño, dependencias entre módulos, y orden de implementación. Es el blueprint que seguirá Claude.
5	<code>/speckit-checklist</code> (opcional)	Valida que el plan esté completo, claro y consistente antes de dividirlo en tareas. Genera una lista de verificación de completitud y coherencia.
6	<code>/speckit-tasks</code>	Divide el plan en tareas accionables, ordenadas y con especificaciones exactas. Cada tarea es lo suficientemente pequeña para implementarse en una sola sesión.
7	<code>/speckit-analyze</code> (opcional)	Revisión de consistencia cross-artefacto: verifica que la spec, el plan y las tareas estén alineadas entre sí. Detecta contradicciones o gaps antes de implementar.
8	<code>/speckit-implement</code>	Ejecuta la implementación siguiendo las tareas definidas. Aquí es donde security-guidance y superpowers corren automáticamente en background, asegurando calidad y seguridad.

4.2 Skills de Git (spec-kit)

Spec-kit también incluye skills especializadas para gestión de git, que siguen las convenciones definidas en la constitution:

Skill	Qué hace
/speckit-git-initialize	Inicializa el repo git con la configuración correcta del proyecto
/speckit-git-feature	Crea una branch de feature siguiendo las convenciones del proyecto
/speckit-git-commit	Genera mensajes de commit semánticos basados en los cambios
/speckit-git-validate	Valida que los cambios cumplan con las convenciones antes de commitear
/speckit-git-remote	Configura el remote del repositorio
/speckit-taskstoissues	Convierte las tareas de spec-kit en issues de GitHub
/speckit-agent-context-update	Actualiza el contexto del agente con el estado actual del proyecto

5. Skills Globales: Disponibles en Cualquier Proyecto

Estas skills están disponibles en TODAS tus sesiones de Claude Code, sin importar el proyecto. No requieren setup adicional.

5.1 /ui-ux-pro-max — Diseño de Interfaces

La skill más poderosa para trabajo visual. Tiene conocimiento de 67 estilos de diseño, 96 paletas de colores, 57 combinaciones de fuentes, 25 tipos de gráficas y 13 stacks tecnológicos. Es como tener un diseñador UI/UX senior disponible 24/7.

Capacidad	Ejemplos de uso
67 Estilos de diseño	glassmorphism, claymorphism, neumorphism, brutalism, minimalismo, bento grid, dark mode, skeuomorphism, flat design, material design...
96 Paletas de colores	Desde paletas corporativas hasta vibrantes y experimentales, con variantes light/dark y accesibilidad WCAG
57 Combinaciones de fuentes	Pairings probados de Google Fonts: serif+sans, display+body, monospace+humanist, etc.
13 Stacks tecnológicos	React, Next.js, Vue, Svelte, SwiftUI, React Native, Flutter, Tailwind CSS, shadcn/ui, etc.
25 Tipos de gráficas	Line, bar, pie, scatter, heatmap, treemap, funnel, gauge, candlestick, radar, sankey...
Acciones disponibles	plan, build, create, design, implement, review, fix, improve, optimize, enhance, refactor, check

Ejemplos de cómo usarlo:

```
"Usa /ui-ux-pro-max para crear un hero section glassmorphism en Next.js"

"Usa /ui-ux-pro-max para revisar el navbar y hacerlo responsive"

"Usa /ui-ux-pro-max para diseñar un dashboard dark mode con bento grid"

"Usa /ui-ux-pro-max para crear una paleta de colores para SeenLabs"
```

5.2 Skills de Ingeniería

Skill	Cuándo usarla	Qué produce
/engineering:debug	Algo no funciona y no sabes por qué	Análisis sistemático de causa raíz, hipótesis ordenadas por probabilidad, pasos para reproducir y verificar el fix
/engineering:architecture	Antes de construir algo nuevo de importancia	Diagrama de componentes, decisiones de diseño documentadas, trade-offs evaluados
/engineering:system-design	Diseñar sistemas complejos (APIs, bases de datos, microservicios)	Diseño completo: escalabilidad, consistencia, latencia, componentes y sus interacciones
/engineering:tech-debt	Identificar qué partes del código frenan el desarrollo	Inventario de deuda técnica priorizado por impacto y esfuerzo
/engineering:testing-strategy	Definir cómo testear el proyecto	Estrategia de testing: unit, integration, e2e, qué mockear, qué no
/engineering:deploy-checklist	Antes de hacer deploy a producción	Checklist personalizada: env vars, migraciones, rollback plan, monitoreo
/engineering:documentation	Documentar código, APIs o arquitectura	Documentación estructurada y mantenible
/engineering:code-review	Revisar código propio o de terceros	Análisis de correctitud, seguridad, performance y mantenibilidad
/engineering:incident-response	Cuando hay un incidente en producción	Protocolo de respuesta, timeline, comunicación y post-mortem
/engineering:standup	Generar update de standup	Resumen de lo trabajado, bloqueadores y próximos pasos

5.3 Skills de Marketing y Contenido

Skill	Qué hace
/marketing:content-creation	Crea contenido completo para web, blog, redes sociales. Define tono, estructura y CTA.
/marketing:draft-content	Borrador rápido de piezas de contenido específicas (un post, una sección, un copy).
/marketing:brand-review	Revisa la consistencia de marca: tono de voz, colores, tipografía, mensajes clave.
/marketing:seo-audit	Auditoría SEO: keywords, meta tags, estructura de headings, velocidad, links internos.
/marketing:campaign-plan	Plan completo de campaña de marketing con objetivos, canales, mensajes y métricas.

/marketing:email-sequen ce	Secuencia de emails de onboarding, nurturing o conversión con copy y timing.
/marketing:competitive- brief	Brief competitivo: análisis de competidores, posicionamiento y oportunidades.
/marketing:performance- report	Reporte de performance de una campaña o canal con insights y recomendaciones.

5.4 Skills de Procesamiento de Archivos

Skill	Formatos	Qué puede hacer
/anthropic-skills :pdf	.pdf	Leer, extraer texto/tablas, combinar, dividir, rotar páginas, agregar marcas de agua, crear PDFs, llenar formularios, encriptar, OCR en escaneados
/anthropic-skills :xlsx	.xlsx .xlsm .csv .tsv	Abrir, leer, editar, agregar columnas, calcular fórmulas, formatear, crear gráficas, limpiar datos sucios, convertir entre formatos tabulares
/anthropic-skills :docx	.docx	Crear y editar documentos Word, formatear, agregar tablas/imágenes, combinar documentos, extraer texto
/anthropic-skills :pptx	.pptx	Crear presentaciones PowerPoint, agregar slides, formatear, insertar gráficas e imágenes, modificar templates

5.5 Skills de Datos y Análisis

Skill	Qué hace
/data:analyze	Análisis exploratorio completo de un dataset con estadísticas descriptivas e insights.
/data:explore-data	Exploración inicial: shape, tipos, nulos, duplicados, distribuciones básicas.
/data:sql-queries	Escribe queries SQL optimizadas para cualquier base de datos relacional.
/data:write-query	Genera queries específicas a partir de una descripción en lenguaje natural.
/data:statistical-analysi s	Análisis estadístico: correlaciones, regresiones, tests de hipótesis, intervalos de confianza.
/data:validate-data	Valida calidad de datos: integridad, consistencia, outliers, valores inválidos.
/data:create-viz	Crea visualizaciones de datos apropiadas para el tipo de análisis.
/data:data-visualization	Diseña dashboards y visualizaciones interactivas completas.
/data:build-dashboard	Construye un dashboard funcional con métricas clave e indicadores.

/data:data-context-extractor

Extrae contexto y metadatos de una fuente de datos para entenderla mejor.

6. Flujos de Trabajo Completos

6.1 Construir una Feature Nueva

```
# 1. Define qué quieres construir

/speckit-specify

# 2. Aclara ambigüedades (si las hay)

/speckit-clarify

# 3. Genera el plan técnico

/speckit-plan

# 4. Valida el plan (opcional)

/speckit-checklist

# 5. Divide en tareas

/speckit-tasks

# 6. Si hay UI involucrada

/ui-ux-pro-max → diseña los componentes primero

# 7. Implementa (security-guidance + superpowers corren automático)

/speckit-implement

# 8. Revisa el código generado

/code-review
```

```
# 9. Revisión de seguridad del branch
```

```
/security-review
```

```
# 10. Antes de hacer deploy
```

```
/engineering:deploy-checklist
```

6.2 Resolver un Bug

```
# 1. Describe el bug con contexto
```

```
/engineering:debug
```

```
# 2. Implementa el fix
```

```
→ Claude propone solución con superpowers
```

```
→ security-guidance revisa automáticamente
```

```
# 3. Revisa el fix
```

```
/code-review
```

6.3 Crear Contenido para la Web

```
# 1. Revisa la marca primero
```

```
/marketing:brand-review
```

```
# 2. Crea el contenido
```

```
/marketing:content-creation → textos largos (landing, about, servicios)
```

```
/marketing:draft-content → piezas cortas (headlines, CTAs, microcopy)
```

```
# 3. Optimiza para SEO

/marketing:seo-audit


# 4. Implementa el contenido en la web

/ui-ux-pro-max → componentes visuales

/speckit-implement → integración al código
```

7. Setup Completo en un Proyecto Nuevo

Sigue estos pasos en orden cada vez que inicies un proyecto nuevo.

Paso 1: Herramientas globales (solo 1 vez por máquina)

```
# Instalar uv (gestor de paquetes Python)

curl -LsSf https://astral.sh/uv/install.sh | sh


# Instalar spec-kit CLI

export PATH="$HOME/.local/bin:$PATH"

uv tool install specify-cli --from git+https://github.com/github/spec-kit.git


# Instalar claude-mem

npx claude-mem install
```

Paso 2: Iniciar worker de claude-mem (cada sesión)

```
npx claude-mem start


# Worker corriendo en http://localhost:37702
```

Paso 3: Inicializar spec-kit en el proyecto (1 vez por proyecto)

```
cd /ruta/de/tu/proyecto

specify init . --integration claude --force
```

Paso 4: Crear .claude/settings.json (1 vez por proyecto)

```
# Crear archivo .claude/settings.json con este contenido:

{

  "enabledPlugins": {

    "security-guidance@claude-plugins-official": true,

    "superpowers@claude-plugins-official": true

  }

}
```

Paso 5: Establecer la constitución (1 vez por proyecto)

```
# En la sesión de Claude Code:

/speckit-constitution

# Define: stack, convenciones, restricciones, estándares
```

8. Referencia Rápida — Cheat Sheet

Quiero...	Skill / Comando
Definir qué construir	/speckit-specify
Planificar la implementación	/speckit-plan
Dividir en tareas	/speckit-tasks
Implementar	/speckit-implement
Diseñar UI o componente visual	/ui-ux-pro-max
Debuggear un bug	/engineering:debug
Revisar código que escribió Claude	/code-review
Revisar seguridad del branch	/security-review
Simplificar código existente	/simplify
Diseñar arquitectura del sistema	/engineering:architecture
Antes de hacer deploy	/engineering:deploy-checklist
Definir estrategia de tests	/engineering:testing-strategy
Identificar deuda técnica	/engineering:tech-debt
Escribir contenido para la web	/marketing:content-creation
Auditoría SEO	/marketing:seo-audit
Revisar consistencia de marca	/marketing:brand-review
Crear campaña de marketing	/marketing:campaign-plan
Trabajar con un PDF	/anthropic-skills:pdf
Trabajar con Excel/CSV	/anthropic-skills:xlsx
Trabajar con Word	/anthropic-skills:docx
Trabajar con PowerPoint	/anthropic-skills:pptx
Analizar datos	/data:analyze
Escribir queries SQL	/data:sql-queries
Crear visualizaciones	/data:create-viz
Ver memoria del proyecto	http://localhost:37702

Ingestar el repo en memoria	/learn-codebase
-----------------------------	-----------------